Edge Intelligence

Lab Task 1 - Data Formats - Log file loading – Features

Name : Soni Samarth

Enroll : 25MML0004

1st_lab

[1]:
```
!pip install librosa matplotlib soundfile requests tqdm
```

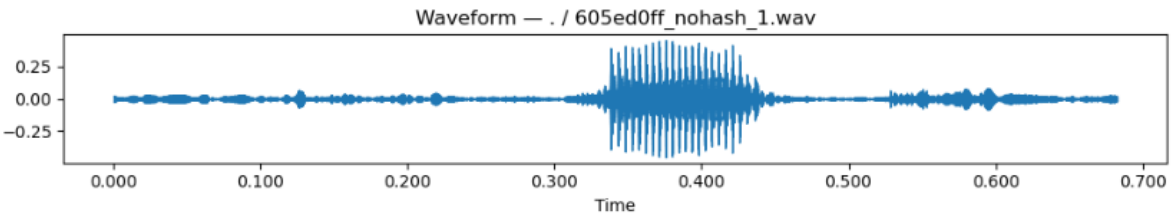
```
# Stream Google Speech Commands v0.02 and inspect files
# Loading all the libraries
import requests, tarfile, io, os
import soundfile as sf
import numpy as np
import librosa
import librosa.display
import matplotlib.pyplot as plt
from IPython.display import Audio, display
from tqdm.auto import tqdm
import random

# URL for Speech Commands v0.02
SC_URL = "https://storage.googleapis.com/download.tensorflow.org/data/speech_commands_v0.02.tar.gz"
```

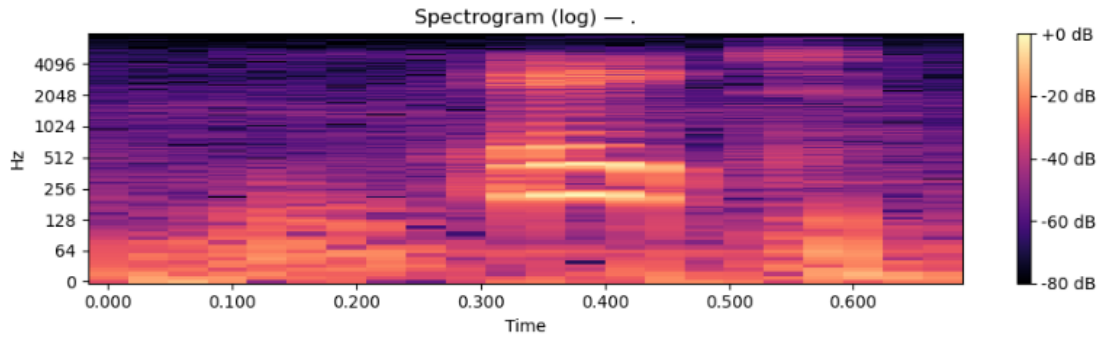
```
def stream_speech_commands(max_files=None, include_words=None):
    """
    Stream the speech-commands tar.gz and yield (member_name, y, sr, label)
      - max_files: stop after this many yielded files (None => all)
      - include_words: list/set of labels to include (None => all)
    """
    resp = requests.get(SC_URL, stream=True)
    resp.raise_for_status()
    # r|gz mode reads entries sequentially from the compressed stream
    with tarfile.open(fileobj=resp.raw, mode='r|gz') as tar:
        count = 0
        for member in tar:
            if not member.isfile():
                continue
            # ignore non-wav
            if not member.name.lower().endswith('.wav'):
                continue
            # member name like: 'yes/0a7c2a8d_nohash_0.wav' or '_background_noise_/...'
            label = member.name.split('/')[0]
            if include_words and label not in include_words:
                continue
            # skip background_noise or other special folders if desired
            # read member bytes
            f = tar.extractfile(member)
            if f is None:
                continue
            data = f.read()
            bio = io.BytesIO(data)
            try:
                # use soundfile to preserve sr and channels
                with sf.SoundFile(bio) as sf_file:
                    y = sf_file.read(dtype='float32')
                    sr = sf_file.samplerate
            except Exception:
                # fallback to librosa (auto resample to sr=None keeps original sr)
                bio.seek(0)
                y, sr = librosa.load(bio, sr=None)
            # make mono
            if y.ndim > 1:
                y = np.mean(y, axis=1)
            yield member.name, y, sr, label
            count += 1
            if max_files is not None and count >= max_files:
                break
```

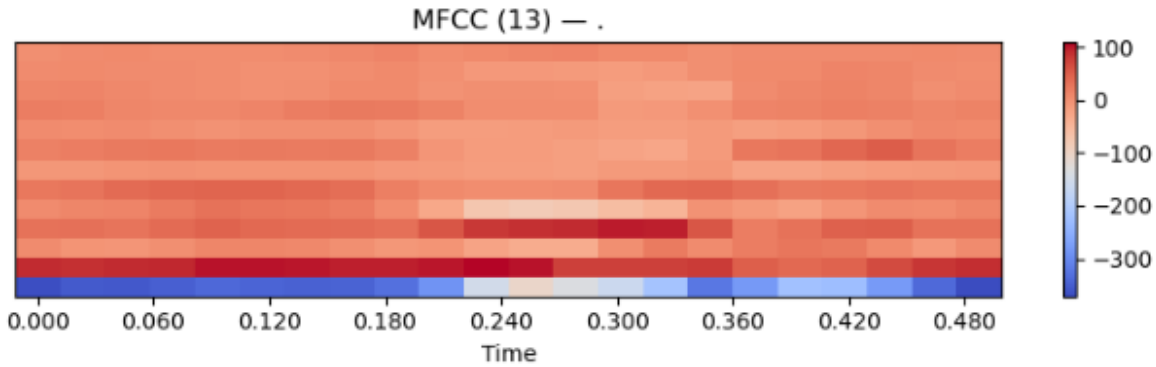
```python
# Example: inspect 4 random files from 4 labels
sample_labels = ["yes", "no", "up", "down"]
print("Sampling a few files (may take a moment while streaming)...\n")
gen = stream_speech_commands(max_files=200)  # stream some items to choose randomly
buffer = list(gen)  # small buffer (200 files)
# pick one from each label if present
chosen = []
for lbl in sample_labels:
    for item in buffer:
        if item[3] == lbl:
            chosen.append(item)
            break
# fallback: if not found, pick random
while len(chosen) < 4 and buffer:
    chosen.append(random.choice(buffer))

for path, y, sr, label in chosen:
    print("Path:", path, "| Label:", label, "| SR:", sr, "| Duration:", round(len(y)/sr, 3), "s")
    display(Audio(y, rate=sr))
    # waveform
    plt.figure(figsize=(10,2))
    librosa.display.waveshow(y, sr=sr)
    plt.title(f"Waveform — {label} / {os.path.basename(path)}")
    plt.tight_layout()
    plt.show()
    # spectrogram
    D = librosa.amplitude_to_db(np.abs(librosa.stft(y)), ref=np.max)
    plt.figure(figsize=(10,3))
    librosa.display.specshow(D, sr=sr, x_axis='time', y_axis='log')
    plt.colorbar(format="%+2.0f dB")
    plt.title("Spectrogram (log) — " + label)
    plt.tight_layout()
    plt.show()
    # MFCC
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
    plt.figure(figsize=(8,2.5))
    librosa.display.specshow(mfcc, x_axis='time')
    plt.colorbar()
    plt.title("MFCC (13) — " + label)
    plt.tight_layout()
    plt.show()
```

```
Sampling a few files (may take a moment while streaming)...

Path: ./eight/605ed0ff_nohash_1.wav | Label: . | SR: 16000 | Duration: 0.683 s
```

0:00 / 0:00

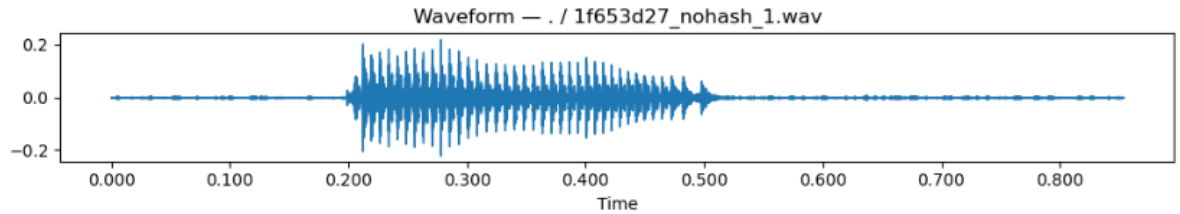


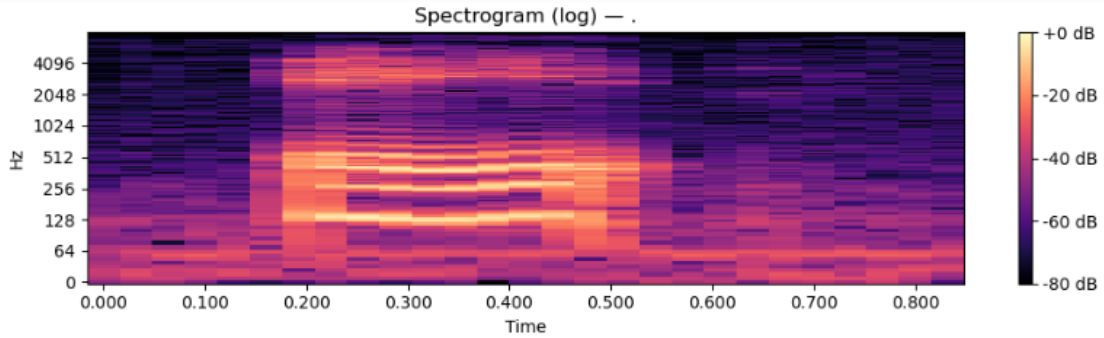


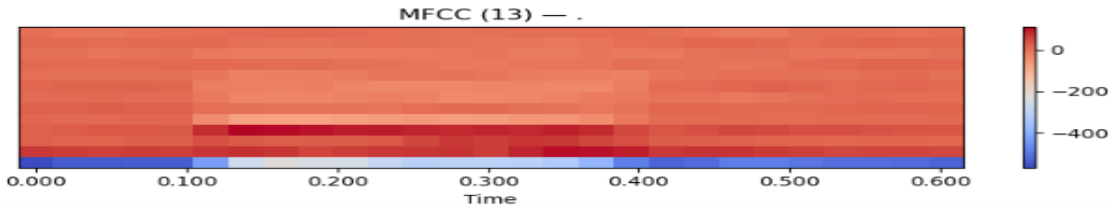


Path: ./eight/1f653d27_nohash_1.wav | Label: . | SR: 16000 | Duration: 0.853 s

0:00 / 0:00







Errors :

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[3], line 6
      4 import soundfile as sf
      5 import numpy as np
----> 6 import librosa
      7 import librosa.display
      8 import matplotlib.pyplot as plt

ModuleNotFoundError: No module named 'librosa'
```

Fix Code

The error ModuleNotFoundError: No module named 'librosa' occurs because the Python environment currently being used does not have the **librosa** library installed. The program attempts to import librosa for audio processing, but since the module is unavailable in the active Jupyter Notebook kernel, Python raises this error and stops execution.

```
---------------------------------------------------------------------------
TimeoutError                              Traceback (most recent call last)
File ~\anaconda3\envs\ml_lab\Lib\site-packages\urllib3\connection.py:198, in HTTPConnection._new_conn(self)
    197 try:
--> 198     sock = connection.create_connection(
    199         (self._dns_host, self.port),
    200         self.timeout,
    201         source_address=self.source_address,
    202         socket_options=self.socket_options,
    203     )
    204 except socket.gaierror as e:

File ~\anaconda3\envs\ml_lab\Lib\site-packages\urllib3\util\connection.py:85, in create_connection(address, timeout, source_address, socket_options)
     84 try:
---> 85     raise err
     86 finally:
     87     # Break explicitly a reference cycle
```

The TimeoutError occurs when the program attempts to establish a network connection but does not receive a response within the specified time limit. In this case, the error originates from the urllib3 library while trying to create an HTTP connection, indicating that the system was unable to connect to the external server.

```
    700 start = preferred_clock()
    702 # Send the request
--> 703 r = adapter.send(request, **kwargs)
    705 # Total elapsed time of the request (approximately)
    706 elapsed = preferred_clock() - start

File ~\anaconda3\envs\ml_lab\Lib\site-packages\requests\adapters.py:665, in HTTPAdapter.send(self, request, stream, timeout, verify, cert, proxies)
    662 if isinstance(e.reason, ConnectTimeoutError):
    663     # TODO: Remove this in 3.0.0: see #2811
    664     if not isinstance(e.reason, NewConnectionError):
--> 665         raise ConnectTimeout(e, request=request)
    667 if isinstance(e.reason, ResponseError):
    668     raise RetryError(e, request=request)

ConnectTimeout: HTTPConnectionPool(host='opihi.cs.uvic.ca', port=80): Max retries exceeded with url: /sound/genres.tar.gz (Caused by ConnectTimeoutErro
r(<urllib3.connection.HTTPConnection object at 0x000001DE1EB02270>, 'Connection to opihi.cs.uvic.ca timed out. (connect timeout=None)'))
```

Fix Code

The error indicates a **connection timeout** that occurred while the Python program was trying to download a file from an external server (opihi.cs.uvic.ca) using the requests library. A connection timeout happens when the client sends a request to a server but does not receive any response within the allowed time limit.

NLTK :

LAB_2

[1]:
```
pip install nltk
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: nltk in c:\programdata\anaconda3\lib\site-
packages (3.7)
Requirement already satisfied: regex>=2021.8.3 in
c:\programdata\anaconda3\lib\site-packages (from nltk) (2022.7.9)
Requirement already satisfied: click in c:\programdata\anaconda3\lib\site-
packages (from nltk) (8.0.4)
Requirement already satisfied: tqdm in c:\programdata\anaconda3\lib\site-
packages (from nltk) (4.64.1)
Requirement already satisfied: joblib in
c:\users\batch1\appdata\roaming\python\python39\site-packages (from nltk)
(1.4.2)
Requirement already satisfied: colorama in
c:\users\batch1\appdata\roaming\python\python39\site-packages (from click->nltk)
(0.4.6)
Note: you may need to restart the kernel to use updated packages.

WARNING: Ignoring invalid distribution -ensorflow-intel
(c:\users\batch1\appdata\roaming\python\python39\site-packages)
WARNING: Ignoring invalid distribution -ensorflow-intel
(c:\users\batch1\appdata\roaming\python\python39\site-packages)
WARNING: Ignoring invalid distribution -ensorflow-intel
(c:\users\batch1\appdata\roaming\python\python39\site-packages)
WARNING: Ignoring invalid distribution -ensorflow-intel
(c:\users\batch1\appdata\roaming\python\python39\site-packages)
WARNING: Ignoring invalid distribution -ensorflow-intel
(c:\users\batch1\appdata\roaming\python\python39\site-packages)
WARNING: Ignoring invalid distribution -ensorflow-intel
(c:\users\batch1\appdata\roaming\python\python39\site-packages)
```

[2]:
```
import nltk
```

[3]:
```
nltk.download('names')
```

```
[nltk_data] Downloading package names to
[nltk_data]     C:\Users\batch1\AppData\Roaming\nltk_data…
[nltk_data]   Unzipping corpora\names.zip.
```

[3]: True

[29]:
```
def gender_features(word):
    return {'last_letter': word[-1]}
```

[30]:
```
from nltk.corpus import names
names.words()
print(len(names.words()))
```

7944

[31]:
```
labelled_name = ([(name, 'male') for name in names.words('male.txt')])+([(name,␣
↪'female') for name in names.words('female.txt')])
```

[32]:
```
print(labelled_name)
```

```
[('Aamir', 'male'), ('Aaron', 'male'), ('Abbey', 'male'), ('Abbie', 'male'),
('Abbot', 'male'), ('Abbott', 'male'), ('Abby', 'male'), ('Abdel', 'male'),
('Abdul', 'male'), ('Abdulkarim', 'male'), ('Abdullah', 'male'), ('Abe',
'male'), ('Abel', 'male'), ('Abelard', 'male'), ('Abner', 'male'), ('Abraham',
'male'), ('Abram', 'male'), ('Ace', 'male'), ('Adair', 'male'), ('Adam',
'male'), ('Adams', 'male'), ('Addie', 'male'), ('Adger', 'male'), ('Aditya',
'male'), ('Adlai', 'male'), ('Adnan', 'male'), ('Adolf', 'male'), ('Adolfo',
'male'), ('Adolph', 'male'), ('Adolphe', 'male'), ('Adolpho', 'male'),
('Adolphus', 'male'), ('Adrian', 'male'), ('Adrick', 'male'), ('Adrien',
'male'), ('Agamemnon', 'male'), ('Aguinaldo', 'male'), ('Aguste', 'male'),
('Agustin', 'male'), ('Aharon', 'male'), ('Ahmad', 'male'), ('Ahmed', 'male'),
('Ahmet', 'male'), ('Ajai', 'male'), ('Ajay', 'male'), ('Al', 'male'), ('Alaa',
'male'), ('Alain', 'male'), ('Alan', 'male'), ('Alasdair', 'male'), ('Alastair',
'male'), ('Albatros', 'male'), ('Albert', 'male'), ('Alberto', 'male'),
('Albrecht', 'male'), ('Alden', 'male'), ('Aldis', 'male'), ('Aldo', 'male'),
('Aldric', 'male'), ('Aldrich', 'male'), ('Aldus', 'male'), ('Aldwin', 'male'),
('Alec', 'male'), ('Aleck', 'male'), ('Alejandro', 'male'), ('Aleks', 'male'),
('Aleksandrs', 'male'), ('Alessandro', 'male'), ('Alex', 'male'), ('Alexander',
'male'), ('Alexei', 'male'), ('Alexis', 'male'), ('Alf', 'male'), ('Alfie',
'male'), ('Alfonse', 'male'), ('Alfonso', 'male'), ('Alfonzo', 'male'),
('Alford', 'male'), ('Alfred', 'male'), ('Alfredo', 'male'), ('Algernon',
'male'), ('Ali', 'male'), ('Alic', 'male'), ('Alister', 'male'), ('Alix',
'male'), ('Allah', 'male'), ('Allan', 'male'), ('Allen', 'male'), ('Alley',
'male'), ('Allie', 'male'), ('Allin', 'male'), ('Allyn', 'male'), ('Alonso',
'male'), ('Alonzo', 'male'), ('Aloysius', 'male'), ('Alphonse', 'male'),
('Alphonso', 'male'), ('Alston', 'male'), ('Alton', 'male'), ('Alvin', 'male'),
('Alwin', 'male'), ('Amadeus', 'male'), ('Ambros', 'male'), ('Ambrose', 'male'),
('Ambrosi', 'male'), ('Ambrosio', 'male'), ('Ambrosius', 'male'), ('Amery',
'male'), ('Amory', 'male'), ('Amos', 'male'), ('Anatol', 'male'), ('Anatole',
'male'), ('Anatollo', 'male'), ('Anatoly', 'male'), ('Anders', 'male'),
('Andie', 'male'), ('Andonis', 'male'), ('Andre', 'male'), ('Andrea', 'male'),
('Andreas', 'male'), ('Andrej', 'male'), ('Andres', 'male'), ('Andrew', 'male'),
```

```
('Waly', 'female'), ('Wanda', 'female'), ('Wandie', 'female'), ('Wandis',
'female'), ('Waneta', 'female'), ('Wenda', 'female'), ('Wendeline', 'female'),
('Wendi', 'female'), ('Wendie', 'female'), ('Wendy', 'female'), ('Wenona',
'female'), ('Wenonah', 'female'), ('Whitney', 'female'), ('Wileen', 'female'),
('Wilhelmina', 'female'), ('Wilhelmine', 'female'), ('Wilie', 'female'),
('Willa', 'female'), ('Willabella', 'female'), ('Willamina', 'female'),
('Willetta', 'female'), ('Willette', 'female'), ('Willi', 'female'), ('Willie',
'female'), ('Willow', 'female'), ('Willy', 'female'), ('Willyt', 'female'),
('Wilma', 'female'), ('Wilmette', 'female'), ('Wilona', 'female'), ('Wilone',
'female'), ('Wilow', 'female'), ('Windy', 'female'), ('Wini', 'female'),
('Winifred', 'female'), ('Winna', 'female'), ('Winnah', 'female'), ('Winne',
'female'), ('Winni', 'female'), ('Winnie', 'female'), ('Winnifred', 'female'),
('Winny', 'female'), ('Winona', 'female'), ('Winonah', 'female'), ('Wren',
'female'), ('Wrennie', 'female'), ('Wylma', 'female'), ('Wynn', 'female'),
('Wynne', 'female'), ('Wynnie', 'female'), ('Wynny', 'female'), ('Xaviera',
'female'), ('Xena', 'female'), ('Xenia', 'female'), ('Xylia', 'female'),
('Xylina', 'female'), ('Yalonda', 'female'), ('Yehudit', 'female'), ('Yelena',
'female'), ('Yetta', 'female'), ('Yettie', 'female'), ('Yetty', 'female'),
('Yevette', 'female'), ('Yoko', 'female'), ('Yolanda', 'female'), ('Yolande',
'female'), ('Yolane', 'female'), ('Yolanthe', 'female'), ('Yonina', 'female'),
('Yoshi', 'female'), ('Yoshiko', 'female'), ('Yovonnda', 'female'), ('Yvette',
'female'), ('Yvonne', 'female'), ('Zabrina', 'female'), ('Zahara', 'female'),
('Zandra', 'female'), ('Zaneta', 'female'), ('Zara', 'female'), ('Zarah',
'female'), ('Zaria', 'female'), ('Zarla', 'female'), ('Zea', 'female'),
('Zelda', 'female'), ('Zelma', 'female'), ('Zena', 'female'), ('Zenia',
'female'), ('Zia', 'female'), ('Zilvia', 'female'), ('Zita', 'female'),
('Zitella', 'female'), ('Zoe', 'female'), ('Zola', 'female'), ('Zonda',
'female'), ('Zondra', 'female'), ('Zonnya', 'female'), ('Zora', 'female'),
('Zorah', 'female'), ('Zorana', 'female'), ('Zorina', 'female'), ('Zorine',
'female'), ('Zsa Zsa', 'female'), ('Zsazsa', 'female'), ('Zulema', 'female'),
('Zuzana', 'female')]
```

[33]:
```
import random
random .shuffle(labelled_name)
featuresets = [(gender_features(n) , gender) for (n,gender) in labelled_name]

train_set, test_set = featuresets[5000:] , featuresets[:2000]

import nltk
classifier = nltk.NaiveBayesClassifier.train(train_set)
classifier.classify(gender_features('David'))
classifier.classify(gender_features('Yann Lecun'))
print(nltk.classify.accuracy(classifier , test_set))
```

```
0.76
```

[23]:
```
import pandas as pd
```

```
df = pd.read_csv('C:/Users/batch1/Downloads/logs_dataset.csv')
```

[24]:
```
df.head()
```

[24]:

	@timestamp	_id	ip_address
0	July 8th 2019, 14:43:03.000	XswJOmsBoTGddM7vxMDB	10.1.1.285
1	July 8th 2019, 14:43:01.000	dKQJOmsB7mPOGwVzvJjz	10.1.2.389
2	July 8th 2019, 14:42:59.000	CcwJOmsBoTGddM7vtb8y	10.1.1.415
3	July 8th 2019, 14:42:57.000	bKQJOmsB7mPOGwVzrZdT	10.1.1.79
4	July 8th 2019, 14:42:55.000	L6QJOmsB7mPOGwVzpZeI	10.1.1.60

[ ]:

Log Data Analytics :

LAB_3

[1]:
```
import pandas as pd
import numpy as np
```

[5]:
```
df = pd.read_csv("/content/logs_dataset.csv")
```

[6]:
```
df.head()
```

[6]:

	@timestamp	_id	ip_address
0	July 8th 2019, 14:43:03.000	XswJ0msBoTGddM7vxMDB	10.1.1.285
1	July 8th 2019, 14:43:01.000	dKQJ0msB7mP0GwVzvJjz	10.1.2.389
2	July 8th 2019, 14:42:59.000	CcwJ0msBoTGddM7vtb8y	10.1.1.415
3	July 8th 2019, 14:42:57.000	bKQJ0msB7mP0GwVzrZdT	10.1.1.79
4	July 8th 2019, 14:42:55.000	L6QJ0msB7mP0GwVzpZeI	10.1.1.60

[7]:
```
df.describe()
```

[7]:

	@timestamp	_id	ip_address
count	68126	68126	68125
unique	68096	67254	386
top	July 8th 2019, 13:45:09.000	#NAME?	10.1.1.63
freq	4	873	461

[8]:
```
df['@timestamp'] = (
    df['@timestamp']
    .str.replace(r'(\d+)(st|nd|rd|th)',
                 r'\1',regex=True)
)
df['@timestamp'] = pd.to_datetime(df['@timestamp'],format='%B %d %Y, %H:%M:%S.%f')
```

[9]:
```
df.sort_values(['ip_address','@timestamp'],inplace=True)
```

[10]:
```
df['shift_time'] = df.groupby(['ip_address'])['@timestamp'].shift(1)
```

[11]:
```
df.head()
```

[11]:

	@timestamp	_id	ip_address	shift_time
66012	2019-07-06 16:13:07	X-gPyGsB7mP0GwVzgqch	10.1.1.1	NaT
65956	2019-07-06 16:15:27	rxERyGsBoTGddM7vpBj2	10.1.1.1	2019-07-06 16:13:07
65781	2019-07-06 16:22:45	lukYyGsB7mP0GwVzU0j7	10.1.1.1	2019-07-06 16:15:27
65440	2019-07-06 16:36:57	SxllyGsBoTGddM7vVIsG	10.1.1.1	2019-07-06 16:22:45
65235	2019-07-06 16:45:29	4uotyGsB7mP0GwVzJMog	10.1.1.1	2019-07-06 16:36:57

[12]:
```
df['time_diff'] = (df['@timestamp'] - df['shift_time']).dt.seconds//60
```

[13]:
```
df['date'] = df['@timestamp'].dt.date
```

[14]:
```
df['hour'] = df['@timestamp'].dt.hour
```

[15]:
```
df['weekday'] = df['@timestamp'].dt.weekday
```

[16]:
```
df['is_weekend'] = ((df['weekday'] == 5) | (df['weekday'] == 6)).astype(int)
```

[17]:
```
df['hour_bucket'] = df['hour']//4
```

[18]:
```
ip_addr = 'ip_address'
```

[19]:
```
ip_counts = df.groupby(ip_addr)['@timestamp'].count().reset_index()
```

[20]:
```
daily_counts = df.groupby([ip_addr,'date'])['@timestamp'].count().reset_index()
```

[21]:
```
daily_counts = daily_counts.rename(columns={'@timestamp':'daily_count'})
```

[22]:
```
daily_counts_avg = daily_counts.groupby(ip_addr)['daily_count'].mean().reset_index()
```

[23]:
```
daily_counts_avg.head()
```

[23]:

	ip_address	daily_count
0	10.1.1.1	46.000000
1	10.1.1.100	93.666667
2	10.1.1.101	37.666667
3	10.1.1.106	43.333333
4	10.1.1.109	49.000000

[24]:
```
weekend_counts = df.groupby([ip_addr,'is_weekend'])['@timestamp'].count().reset_index()
```

[25]:
```
weekend_counts
```

[25]:

	ip_address	is_weekend	@timestamp
0	10.1.1.1	0	50
1	10.1.1.1	1	88
2	10.1.1.100	0	88
3	10.1.1.100	1	193
4	10.1.1.101	0	31
...	...	...	...
767	10.1.2.90	1	181
768	10.1.2.95	0	78
769	10.1.2.95	1	186
770	10.1.2.99	0	41
771	10.1.2.99	1	81

772 rows × 3 columns